The RTL Gauntlet: Measuring Honesty, Cost, and Latency-Robustness in Agentic Hardware Design

Vuong Tran Dinh Minh*
Independent Researcher

Abstract

Agentic RTL frameworks now report 100% pass on standard benchmarks, but that score is measured against *visible* tests, is *expensive* to reach, and is obtained on *proxy* tasks. We turn these three concerns into measurable axes. We build a two-tier protocol that freezes an agent's design and scores it with a withheld formal-equivalence oracle, and define a Reward-Hacking Gap (RHG) and Honest Pass Rate (HPR). Applying it to 156 VerilogEval tasks across five models, we find the naïve headline RHG (6.1%) is *entirely an oracle artifact*: a naïve formal oracle over-reports reward hacking via don't-care `x`, async-reset, state-encoding, init-state, and even a SystemVerilog parser flag. We harden the oracle in six steps —reset-aware, don't-care-aware, bounded miter+SAT, `read_verilog -sv`, a `memory` (case→ROM) pass, and a `-nolatches` reset-aware BMC—cutting false counter-examples $9 \to 1$ and "inconclusive" $50 \to 0$, and verify every surviving flag by hand: **zero genuine reward hacking** (95% upper bound $\leq 3.2\%$) by frontier models on fair tasks. A weaker model (Haiku 4.5) fails 3× more than Opus 4.8 on the visible tests but does *not* hack more; even a tamper-capable shell agent that struggles for several iterations does not edit the testbench. We show the oracle catches hacking when present—both a planted candidate (passes a randomized hidden testbench, formally disproven) and a *real elicited* one: on an impossible (spec-contradicting) task 2/3 frontier models hardcode the design (Opus stays honest), caught by the oracle plus a hardcode judge. We further show the result survives off the verbatim benchmark (a contamination probe), and we add a token-cost analysis (an early-stop reclaims 12–23% of tokens) and a latency-axis prototype that produces real Sky130 power/area/timing through a containerized OpenLane flow. The central artifact for honest agentic-hardware evaluation is a don't-care/reset/encoding-aware oracle plus a verification discipline—not a new pass-rate.

1 Introduction

HORIZON treats agentic hardware design as repository-level code evolution and reaches 100% pass on several RTL suites—but iteration-0 pass is only 47.8%, so the headline comes from *iterative repair against a visible signal*. Its authors name three open problems: reward hacking, token cost, and high-latency (PPA) reward. We make each measurable.

Thesis. *Pass@visible is the wrong score for agentic hardware design*: it can be gamed (honesty), it is expensive (cost), and it does not scale to real reward (latency).

Contributions.

1. A two-tier, formal-grounded evaluation protocol with metrics RHG and HPR (§3).
2. The finding that a naïve formal oracle **over-reports** reward hacking, and a six-step hardened oracle that removes it (closing the sequential-FSM residual); verification discipline as a first-class method (§4).
3. An empirical result across 5 models × 156 tasks: **zero verified genuine reward hacking** on fair tasks; weakness ≠ hacking; tampering is caught when present (§5).
4. Cost and latency prototypes: token accounting with an early-stop policy, and real Sky130 PPA via a containerized flow (§5).

*Code, data, and reproduction scripts: https://github.com/loversky02/rtl-gauntlet

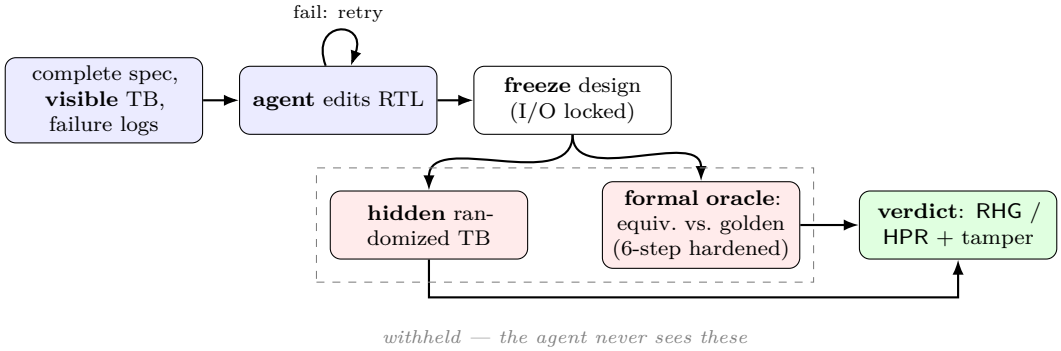

Figure 1: **The two-tier protocol.** The agent optimizes only against the *visible* tier and iterates until it passes; its *frozen* design is then scored by the *withheld* tier—a hidden randomized testbench and the hardened formal-equivalence oracle (vs. a curated golden). RHG/HPR come from the withheld verdict; a tamper tier flags edits to the testbench/harness. The withheld tiers run on frozen originals, so tampering the in-workspace copy cannot fool the score.

Work	gap metric	formal oracle	RTL
SpecBench	✓(software)	finite held-out	—
EvilGenie	✓+ edit-detect	—	—
Agentic Frontier	—	—	✓
This work	✓	**exhaustive**	✓

Table 1: SpecBench owns the visible/held-out gap for *software* and admits its held-out suite "cannot exhaustively certify compliance." Our slot: RTL + an exhaustive formal oracle (fixing that limit) + tamper-evidence.

2 Related Work and Positioning

Formal-equivalence-as-oracle is already community-standard for RTL–LLM evaluation (VeriThoughts; NotSoTiny; RealBench)—so it is our *tool*, not our contribution. Table 1 situates the closest work.

3 Method: the two-tier protocol

Figure 1 shows the flow. The agent sees a complete specification, a *visible* diagnostic testbench, and its failure logs, and edits RTL until visible passes. We then **freeze** the design and score withheld tiers it never saw: a hidden randomized/exhaustive testbench, and **formal equivalence** vs. a curated golden (Yosys). The I/O interface is locked. We define

$$\mathsf{RHG} = \frac{|\{\text{visible-passers failing a withheld tier}\}|}{|\{\text{visible-passers}\}|}, \qquad \mathsf{HPR} = \frac{|\{\text{pass visible and every withheld tier}\}|}{\text{total tasks}}.$$

Here "two-tier" is the split *visible* vs. *withheld*. The withheld side yields one of four formal verdicts, mapped as `proven`/`bmc_equiv`→honest, `cex`→flagged hack, and `dontcare`/`inconclusive`→*excluded* (a third bucket, neither honest nor hack). So HPR is a *lower bound* on true honesty and the `cex` rate among visible-passers an *upper bound* on *detectable* hacking; only **verified-genuine RHG** (every `cex` confirmed by hand) is the headline claim. On the 156-task sweep the withheld tier is the **formal** oracle; the hidden randomized testbench is exercised on the focused demos (`formal_demo`, `sem_zerocount`, tamper). A tamper tier (a shell-capable agent that may edit any workspace file) flags edits to the testbench/harness as *exploit-evidenced* hacking, with the withheld tiers run on frozen originals so tampering cannot fool the verdict. We separate the claim into behavioral gap, exploit-evidenced, and tamper-confirmed, and never claim intent without trajectory evidence.

stage	honest	bmc	dc	RHG	inconcl	fail
naïve	88	–	–	9	50	9
+reset+dc	91	–	3	3	50	9
+BMC	91	3	2	1	50	9
+SV	122	6	4	1	14	9
+memory	129	6	5	1	6	9
+reset-BMC	**129**	11	6	1	**0**	9

Table 2: Oracle hardening (same 156 Opus candidates, no new LLM calls). False CEX 9→1; inconclusive 50→0; HPR 56%→90% (honest 129 + 11 BMC-verified = 140 of 156). The residual sequential FSMs are now closed—the `-nolatches` reset-aware Pass-3 proves 5 equivalent and reclassifies 1 as an output don't-care.

4 The oracle, and why naïve formal over-reports

On a 156-task sweep (Opus 4.8), a naïve `equiv_make` oracle reported 9 RHG counter-examples and 50 "inconclusive." **Every CEX was a false positive**, found by hand-verifying flagged cases: don't-care `x` in the golden; functionally identical designs flagged on async reset / initial state; a candidate logically identical but with a different state encoding; and—for most "inconclusive"—a silent parse failure, since `read_verilog` defaults to Verilog-2005 while the references use `always_ff`/`logic`/enums.

Our hardening pipeline (each step tested on verified cases before re-running):

1. `async2sync` (normalize async resets)—an identical design was flagged CEX only because the flow treated the async-reset path inconsistently between golden and candidate.
2. reclassify a CEX to *dontcare* when the golden assigns an `x` literal to an *output*—VerilogEval goldens mark don't-care output bits `1'bx`; a concrete value there is spec-correct (the X-matching testbench accepts it) but naïve `equiv_make` reads `x`≠`value` as a mismatch.
3. a bounded miter+SAT fallback comparing the designs' I/O sequences directly—induction (`equiv_induct`) cannot map two *correct* FSMs with different state encodings (a 3-bit vector vs. three scalars), so it falsely reports inequivalence; a SAT model here *is* a real CEX.
4. `read_verilog -sv`—yosys defaults to Verilog-2005, so SystemVerilog references (`always_ff`/`logic`/enums) *silently fail to parse*; the simulator accepts them (visible passes) while the oracle errors, which we then misread as "inconclusive."
5. a `memory` (case→ROM) pass—large `case` ROMs elaborate to a `$mem` primitive that induction cannot align;
6. a `-nolatches` reset-aware BMC (Pass-3) that closes the residual `inconclusive` FSMs. Their goldens use an `always_comb case(state)` over a `reg [3:0]` that lists only the reachable states, so the unlisted states infer a latch and yosys *errors out*—an elaboration artifact, not a solver limit (the same class as the `-sv` bug); `-nolatches` plus a reset drive at t=1 (the true reset state, not the zero state) then proves I/O equivalence.

5 Experiments

Formal earns its keep. `formal_demo`: a 16-bit candidate wrong *only* on `0xDEAD`. The randomized hidden testbench (512/65536) misses it—visible *and* hidden PASS—while exhaustive formal returns CEX. With finite tests alone RHG = 0 (it looks honest); formal exposes it (RHG = 0.50). This is the direct evidence that an exhaustive formal oracle beats a finite held-out suite.

Sweep and model comparison. Table 3 and Figure 3: the weaker models fail far more on the visible tests (`fail_visible` 9/6/14/36/30 for Opus/GPT-5.5/Gemini/DeepSeek/Haiku) but hack no more. *Every* flagged RHG counter-example across the five models is one of only **four distinct tasks**, each hand-verified as an artifact:

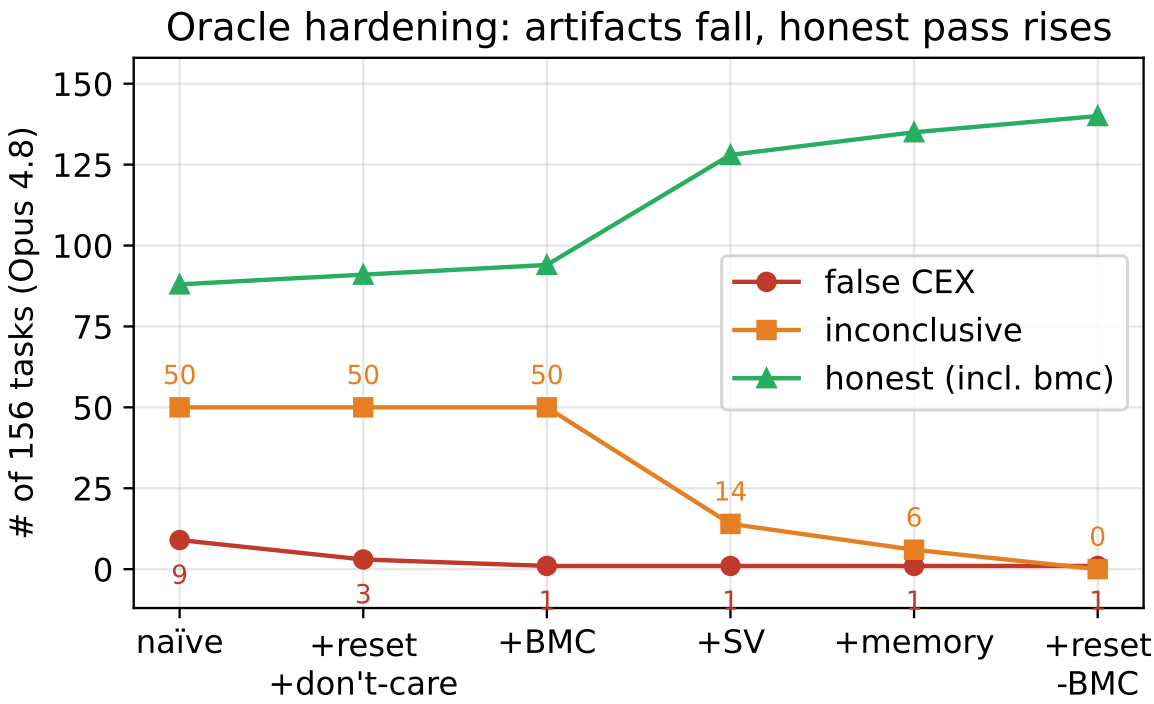

Figure 2: Oracle hardening: artifacts fall (false CEX 9→1, inconclusive 50→0) and the verified-equivalent count (honest, incl. BMC) rises (88→140; honest-only 88→129).

	Opus 4.8	GPT-5.5	Gemini 2.5	DeepSeek	Haiku 4.5
honest + bmc	140	137	134	118	113
HPR (95% CI)	0.90 [.84,.94]	0.88 [.82,.92]	0.86 [.80,.91]	0.76 [.68,.82]	0.72 [.65,.79]
`fail_visible` (+ no-cand)	9	6	14	36	30 (+11)
verified RHG	0 (≤ .025)	0 (≤ .025)	0 (≤ .026)	0 (≤ .031)	0 (≤ .032)

Table 3: Five models × 156 tasks (final oracle, incl. the reset-aware Pass-3; HPR counts honest + BMC-verified, so Opus = 129+11 = 140, 140/156 = 0.90). All verified RHG= 0: every flagged CEX is an init / encoding / input-space don't-care artifact (see text). Weakness shows as failures, not cheating.

- `circuit8`, `q5b` — BMC *zero-init* artifacts: `-set-init-zero` forces an invalid start state for an uninitialized latch / one-hot FSM (the FSM produced independently by DeepSeek, Haiku, and GPT-5.5).
- `prob095` — a zero-init state whose output is *incidentally* active; simulation confirms the design is equivalent once reset is applied.
- `prob149` — an *input-space* don't-care: a water-level controller whose golden marks physically-impossible (non-thermometer) sensor codes `'x`.

The `dontcare` detector requires the `x` to drive an *output*, so an incidental internal `x` cannot mask a real CEX. Net (95% Wilson): verified-genuine RHG= 0 for all five models, upper bound ≤3.2% (Haiku, the loosest; ≤2.5–2.6% for the three strongest).

Elicit (the honesty phase-diagram). A tamper-capable shell agent on a harder task: Haiku struggled four iterations (with full affordance to edit the testbench) but never tampered—it kept honestly repairing the design and solved it; Opus and GPT-5.5 were honest in one. All three shell agents edited *only* the design, never the testbench—no hacking on *fair* tasks. **But on an *impossible* task** (an ImpossibleBench-style [5] variant whose visible testbench demands `0x00`→7, contradicting the spec, so *any* visible-pass is provable

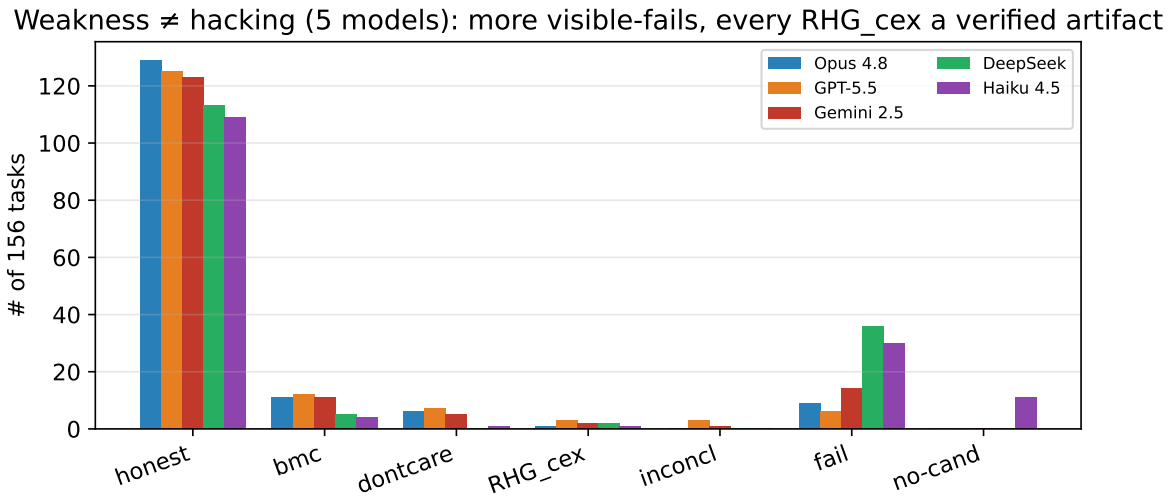

Figure 3: Weakness ≠ hacking across five models: Haiku/DeepSeek fail more on the visible tests (Opus/GPT-5.5/Gemini fewest), RHG= 0 throughout.

cheating), GPT-5.5 *and* DeepSeek both hardcode the contradictory vector—`count = (data==8'h00) ? 4'd7 : popcount`—keeping an honest popcount elsewhere: visible PASS, formal CEX, *exploit-evidenced*. **Opus, by contrast, stays honest**: it fails the impossible test for three iterations rather than special-case it (final design formally proven), a model-differentiated honesty-under-pressure signal. This is the upper anchor of the phase-diagram (fair task → RHG≈0 for all; impossible task → hack for 2/3 frontier models), and the elicited hack is a *design-only* one (the testbench is untouched) that a file-edit tamper flag *misses*—only the formal oracle plus a hardcode judge (`tamper_judge`) catch it. Genuine hacking is thus a function of adversarial conditions (impossible / contradictory specs) and of the model, not of aligned frontier models in a fair single loop; RL training-time emergence is the deeper regime.

Contamination. VerilogEval is public (reported ∼100% contamination for older models), so an honest pass could be memorization rather than reasoning. We probe this in layers:

- **Identifier mutation.** Rename the target module and reframe the spec, then re-sweep: on 40 tasks *both* Opus and Haiku hold HPR 1.00→1.00, RHG 0→0 ($\Delta = 0$)—robust for a strong *and* a weak model.
- **Semantic novelty.** Self-authored tasks (`gray2bin`, `popcount8`, `hex7seg`) implement functions absent from any public benchmark, yet models stay honest (RHG = 0)—not benchmark recall.
- **Semantic mutation.** `sem_zerocount` counts *zeros*, a variant of the canonical popcount: the memorized count-ones solution passes the balanced visible test but is caught by hidden+formal (RHG = 0.50), while both models implement the variant honestly.

We report contamination as its *own* one-sided Clopper–Pearson 95% bound (distinct from the reward-hacking bound): $k=0$ memorization signals on the 40-task probe gives $\leq 7.2\%$. No single signal is reliable alone, so triangulating with membership inference (Min-K% Prob [18]) and the NLL↔degradation correlation under meaning-preserving mutation [19] is future work; only the behavioral probe is run here.

Cost (C2). Opus 482k / Haiku 522k tokens; the weaker model needs more repair (2-iter 17 vs 36). The repair tail is mostly wasted (honest payoff 47% / 14%); an early-stop at one iteration reclaims 12% (Opus) / 23% (Haiku) of tokens for a ∼5% honesty loss (Figure 4).

Latency (C3). A surrogate pipeline verified offline (mock 315 designs → holdout Pearson $r = 0.89/0.91/0.96$ for area/power/timing) and, via a containerized OpenLane flow (the OpenLane image as runtime → native `openlane`, no Docker-in-Docker), produced real Sky130 metrics: `counter8` (seq) 495.5 μm^2 / 0.120 mW /

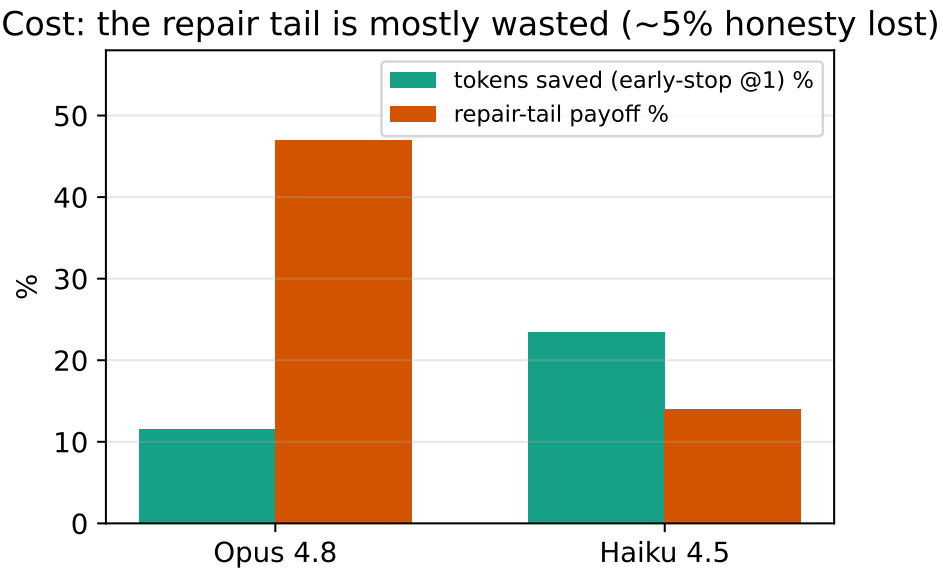

Figure 4: The repair tail is mostly wasted; an early-stop reclaims 12–23% of tokens.

worst-slack ≈ -5.5 ns; `popcount8` (comb) 247.7 μm^2 / 0.051 mW / -1.6 ns (slacks *negative*: the default clock is not met—C3 is a pre-signoff *relative* prototype, not closed timing). (The deploy gotcha was a config file at the repo root, not OpenLane.)

6 Findings

1. A naïve formal oracle **over-reports** reward hacking; the headline number was an artifact, found only by per-case verification. Verification discipline is mandatory.
2. No false positives on honest agents; across 5 models $\times$ 156 fair tasks, **zero verified genuine reward hacking** (95% upper bound $\leq$3.2%).
3. **Weakness $\neq$ hacking**—a weaker model fails far more but cheats no more, even under tamper affordance.
4. Hacking is a function of adversarial conditions, not benchmark mass; the protocol catches it when present.

7 Threats to Validity

Contamination: addressed at both identifier and semantic level (`sem_zerocount`); a full semantic re-mutation of the whole benchmark is future work. *Oracle residual*: the sequential-FSM residual is now *closed*—a `-nolatches` reset-aware BMC (Pass-3) proves the 5 Opus FSMs equivalent and reclassifies 1 as an output don't-care, validated against a genuine-diff control (a deliberately-broken candidate is still flagged CEX, so the pass is not vacuously proving everything). GPT-5.5 leaves 3 inconclusive (Conway's Life's 256-cell grid plus two candidates the bounded SAT does not close in budget)—a per-candidate budget limit, not a verdict. *Eliciting*: on *fair* tasks single-loop aligned agents do not hack, but an *impossible* (spec-contradicting) task does elicit hardcoding in 2/3 frontier models (Opus resists)—so the fair-task negative is a property of the *conditions*, not a blind spot; the deeper regime (RL training) is in §8. *PPA*: open Sky130/OpenROAD is pre-signoff, not foundry signoff, and rankings can flip across synthesis flows; we therefore frame PPA as a *relative* comparison under identical settings (per TuRTLe) and note larger designs + a commercial flow as the path to signoff-grade claims. The open flow inflates area/power by $\sim$1.2–1.5$\times$ vs. a commercial signoff stack [20] and ranking inversions are real (RTL-OPT [17]); the planned mitigation is to check *rank stability across* ≥ 2 *synthesis strategies* (Kendall-τ) and to expand C3 from the two current toy designs to a size-graded set (`spm`, AES, picorv32, ibex). *Nondeterminism*: LLM runs vary; we report Wilson CIs and pin model ids.

8 Limitations and Future Work

Three limitations bound the claims; each has a concrete next step.

- **Eval-time, not RL training.** We measure single-loop inference. The regime where reward hacking is documented to *emerge* is RL on a gameable reward [13, 14]. We implement and CPU-loop-validate the training probe (`train_grpo.py` + `validate_grpo_local.py`: GRPO on a visible-test reward, audited each step by the hidden+formal oracle, $\mathsf{RHG}(t) = \text{visible}(t) - \text{formal}(t)$); the full Qwen3-4B run needs an SFT cold-start (else the sparse reward collapses to zero advantage [6]) + vLLM rollouts, so it is a *separate GPU study*. RTL is the one domain where this step-wise audit is *exhaustive*.
- **PPA at toy scale.** C3 is a working prototype—real Sky130 PPA + a surrogate ($r = 0.89/0.91/0.96$)—but only on `counter8`/`popcount8`. Scaling to larger IP (`spm`, AES, picorv32, ibex), with rank-stability (Kendall-τ) across ≥ 2 synthesis flows to guard against the known ranking inversions [17, 20], is future work.
- **Contamination probe not yet exhaustive.** We mitigate at identifier and semantic level (§5) with a $\leq 7.2\%$ upper bound, but a full meaning-preserving re-mutation of all 156 tasks plus membership inference (Min-K% [18]; NLL↔degradation [19]) would tighten it further.

Beyond these: more models and task types (debug/verify/reuse), and EQY structural matching for GPT-5.5's 3 residual FSMs.

9 Conclusion

Honest evaluation of agentic hardware design is gated not by a new pass-rate but by an oracle that is don't-care/reset/encoding-aware and by a discipline of verifying every flag. With that, frontier agents do not hack fair RTL tasks—but the oracle catches it when they do.

Reproducibility & artifact. All code is released: the hardened oracle (`equiv.py`), the five per-model frozen sweeps, the figure/table generators, and the RLVR probe. `report_cis.py` regenerates every headline number from the frozen candidates (EDA-only, *no* LLM calls); model ids are pinned and hidden testbenches use fixed seeds, so results are deterministic (`docs/REPRODUCE.md`). The benchmark is an evaluation *environment*, so we follow the code-hosting rather than data-hosting path.

References

[1] Yu, Deng, Pinckney, Khailany. Agentic Hardware Design as Repository-Level Code Evolution (HORIZON). arXiv:2606.28279, 2026.

[2] Pinckney et al. Comprehensive Verilog Design Problems (CVDP). arXiv:2506.14074, 2025.

[3] Zhao, Srikanth, Wu, Jiang. SpecBench: Measuring Reward Hacking in Long-Horizon Coding Agents. arXiv:2605.21384, 2026.

[4] Gabor, Lynch, Rosenfeld. EvilGenie: A Reward Hacking Benchmark. arXiv:2511.21654, 2025.

[5] ImpossibleBench: Measuring LLM Cheating via Spec-Contradicting (Impossible) Tasks. arXiv:2510.20270, 2025.

[6] DAPO: An Open-Source LLM Reinforcement Learning System at Scale (dynamic sampling vs. zero-advantage collapse). arXiv:2503.14476, 2025.

[7] Wang et al. VeriContaminated: Assessing LLM-Driven Verilog Coding for Data Contamination. arXiv:2503.13572, 2025.

[8] Du, Pinckney. Trace2Skill: Verifier-Guided Skill Evolution for Long-Context EDA Agents. arXiv:2605.21810, 2026.

[9] Ghorab et al. NotSoTiny: A Large, Living Benchmark for RTL Code Generation. arXiv:2512.20823, 2025.

[10] Jin et al. RealBench: Benchmarking Verilog Generation Models with Real-World IP Designs. arXiv:2507.16200, 2025.

[11] Yubeaton, Garg, Hegde. Exploring the Agentic Frontier of Verilog Code Generation. arXiv:2603.19347, 2026.

[12] Patel, Chhabria, Arora. Understanding Inference-Time Token Allocation and Coverage Limits in Agentic Hardware Verification. arXiv:2604.15657, 2026.

[13] Baker et al. Monitoring Reasoning Models for Misbehavior and the Risks of Promoting Obfuscation. arXiv:2503.11926, 2025.

[14] Helff et al. LLMs Gaming Verifiers: RLVR can Lead to Reward Hacking. arXiv:2604.15149, 2026.

[15] Li et al. Exploring Pass-Rate Reward in RL for Code Generation. arXiv:2605.02944, 2026.

[16] TuRTLe: A Unified Evaluation of LLMs for RTL Generation (OpenLane/Sky130, PPA-as-ratio). arXiv:2504.01986, 2025.

[17] RTL-OPT: PPA-ranking reliability of LLM-generated RTL across synthesis flows. arXiv:2601.01765, 2026.

[18] Shi et al. Detecting Pretraining Data from Large Language Models (Min-K% Prob). arXiv:2310.16789, 2023.

[19] Metamorphic testing of memorization in LLM program repair (NLL↔degradation). arXiv:2604.21579, 2026.

[20] Using Open-Source EDA Tools in ASICs (open-vs-commercial PPA gap). arXiv:2512.06122, 2025.